

C—语义分析器实验报告

一、实验环境

操作系统：Debian 12

编译器：gcc 12.2.0

语法分析器生成工具：Bison 3.8.2

词法分析器生成工具：Flex 2.6.4

二、实验目标

在实验一（词法分析与语法分析）的基础上，实现 C—语言的语义分析与类型检查功能，包括：

- 构建符号表，记录变量、函数、结构体等信息；
- 检查 17 种语义错误（如未定义变量、类型不匹配、重复定义等）；
- 支持可选功能：函数声明、嵌套作用域、结构体结构等价。

三、实现功能概述

本实验实现了 C—语言的语义分析器，在词法分析和语法分析的基础上，对 C—源代码进行语义分析和类型检查。该分析器能够检测 17 种语义错误，并支持函数声明、嵌套作用域和结构等价等扩展功能。

四、附加功能实现

4.1 函数声明（要求 2.1）

扩展语法规则，支持函数声明；

符号表中记录函数的“声明”与“定义”状态；

检查错误类型 18（未定义函数）和 19（声明冲突）。

五、编译与运行说明

5.1 编译方式

使用 make 命令编译项目，需确保已安装 flex、bison、gcc。

5.2 运行方式

`./parser test.c`

六、实验难点

1 类型系统的设计与实现

难点分析：C语言虽然语法简单，但类型系统需要支持基本类型、数组、结构体等复杂类型，特别是多维数组和嵌套结构体的表示。

解决方案：

- 采用递归的链表结构表示复杂类型
- 实现类型深拷贝机制，确保类型信息独立
- 支持名等价和结构等价两种类型判定方式

2 函数声明与定义的协调处理

难点分析：需要区分函数声明与定义，处理多次声明的一致性检查，确保声明与定义匹配。

解决方案：

- 在函数签名中增加 `declared_only` 标志位
- 插入符号表时检查签名一致性
- 维护待处理参数列表，供后续函数体分析使用

七、代码亮点

1. 模块化设计

将系统清晰地划分为多个模块：

- `type.[ch]`：类型系统实现
- `symbol_table.[ch]`：符号表管理
- `semantic.[ch]`：语义分析主逻辑

各模块职责明确，接口清晰，便于维护和扩展。

2. 内存管理严谨

实现了完整的内存管理机制：

- 所有动态分配都通过包装函数进行
- 深拷贝函数确保类型信息的独立性
- 配套的释放函数防止内存泄漏

3. 错误恢复能力强

采用继续分析策略：

- 发现错误后记录并继续分析
- 使用默认类型保证后续分析的进行
- 能够一次性报告多个错误

八、总结

本次实验深入理解了语义分析在编译器中的核心作用，掌握了符号表构建、类型系统设计与错误检查的实现方法。通过模块化设计与递归遍历语法树，我们成功实现了 C 语言的语义分析器，并支持了多项可选功能。尽管实现过程中遇到诸多细节挑战，但最终通过合理的数据结构与算法设计，完成了所有必做与选做要求。